

ML24/25-05 Implement Performance Measurement Experiment

Course: Software Engineering

Lecturer: Damir Dobric / Andreas Pech

Presented by:

Tanvir Ahmed

Matriculation No. # 1386435



OBJECTIVE

The primary goal is to implement a performance measurement experiment in console app to run experiment with variable measurement values which also enables to run any experiment, especially MultiSequence Learning experiment.

TABLE OF CONTENT

- INTRODUCTION
- BACKGROUND
- METHODOLOGY
- IMPLEMENTATION
- RESULTS
- CONCLUSION
- REFERENCE

INTRODUCTION

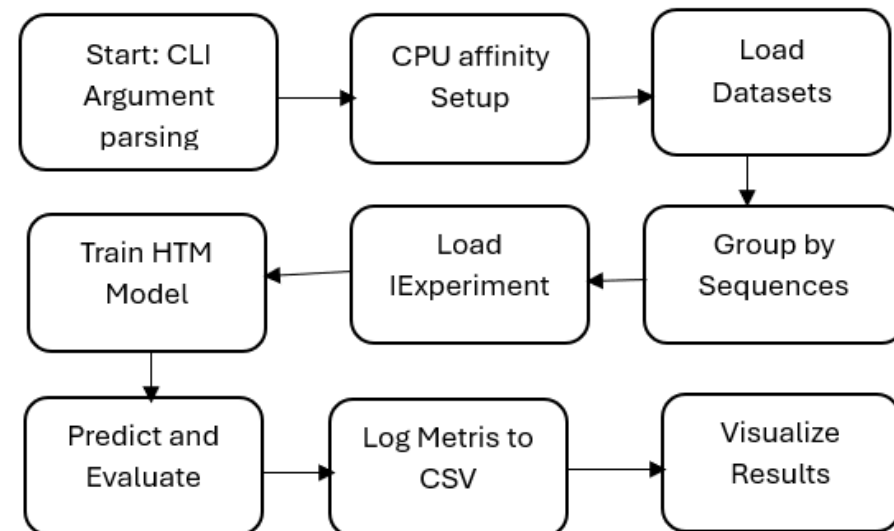
- Multi-sequence learning with HTM entails teaching the algorithm to detect and anticipate patterns in multiple input sequences.
- To perform multi-sequence learning using HTM, we must first encode the input information into Sparse Distributed Representations (SDRs), which can be accomplished with a scalar encoder.
- After the data has been encoded, the spatial pooler generates sparse illustrations of the input sequences that are supplied into the temporal memory element for learning and prediction.
- Using Argument parsing from the command line the experiment runs to calculate the learning time for the variable measurement values.

BACKGROUND

- HIERARCHICAL TEMPORAL MEMORY (HTM)
- SPARSE DISTRIBUTED REPRESENTATION (SDR)
- ENCODER
- SPATIAL POOLER
- TEMPORAL MEMORY

METHODOLOGY

Overview of the experiment:



IMPLEMENTATION

Experiment Configuration and Argument Parsing:

```
private static InputParameter ParseArguments(string[] args)
{
    if (args.Length == 0)
    {
        throw new ArgumentException(" Missing required arguments. Use: dotnet run --sequence-folder <path> --out <csv path> --cores <num> --experiment <class>");
    }

    return new InputParameter
    {
        SequenceFolder = GetArgumentValue(args, "--sequence-folder") ?? throw new ArgumentException(" Missing required argument: --sequence-folder"),
        OutputCsvPath = GetArgumentValue(args, "--out") ?? "output.csv",
        ExperimentClass = GetArgumentValue(args, "--experiment") ?? throw new ArgumentException(" Missing required argument: --experiment"),
        CpuCores = int.Parse(GetArgumentValue(args, "--cores")),
        CpuAffinity = int.Parse(GetArgumentValue(args, "--affinity"))
    };
}
```

```
<TargetFramework>net9.0</TargetFramework>
```

- --sequence-folder <path> --out <output.csv> --cores 1 --affinity 1 --experiment MultiSequenceLearning

IMPLEMENTATION

DATA MODEL:

- The sequence represents the model for processing and storing the dataset.

```
public class Sequence
{ public String name { get; set; }
  public double[] data { get; set; }}
```

```
[
{
  "name": "T1",
  "data": [ 5, 6, 7, 8 ]
},
{
  "name": "T2",
  "data": [ 6, 11, 12, 13 ]
},
{
  "name": "T3",
  "data": [ 1, 2, 3, 4 ]
},
{
  "name": "T4",
  "data": [ 3, 4, 7, 8, 10 ]
}
]
```

```
[
{
  "name": "S1", "data": [0, 2, 5, 7, 8, 11, 13, 14, 17, 21, 23, 24, 25, 26, 27, 28, 29]
},
{
  "name": "S2", "data": [0, 1, 2, 3, 4, 6, 7, 9, 14, 15, 17, 22, 23, 24, 25, 27, 28]
},
{
  "name": "S3", "data": [2, 3, 5, 9, 10, 11, 12, 13, 17, 19, 20, 21, 23, 25, 26, 27, 29]
}
]
```


IMPLEMENTATION:

Dataset Loading:

- LoadDataset()

```
private static List<Sequence> LoadDatasets(string folderPath, string pattern)
{
    List<Sequence> sequences = new List<Sequence>();
    string[] files = Directory.GetFiles(folderPath, pattern);

    foreach (var file in files)
    {
        var seqData = ReadDataset(file);
        sequences.AddRange(seqData);
    }

    return sequences;
}
```

IMPLEMENTATION

IExperiment for loading experiment :

- LoadExperiment()

```
private static IExperiment LoadExperiment(string experimentClassName)
{
    Type experimentType = Type.GetType($"NeocortexApiSamplePerformance.{experimentClassName}");

    if (experimentType == null || !typeof(IExperiment).IsAssignableFrom(experimentType))
        throw new ArgumentException($"Invalid experiment: {experimentClassName}. Make sure it implements IExperiment.");

    return (IExperiment)Activator.CreateInstance(experimentType);
}
```

IMPLEMENTATION

Performance Logging:

- PerformanceLogger.LogPerformance ()

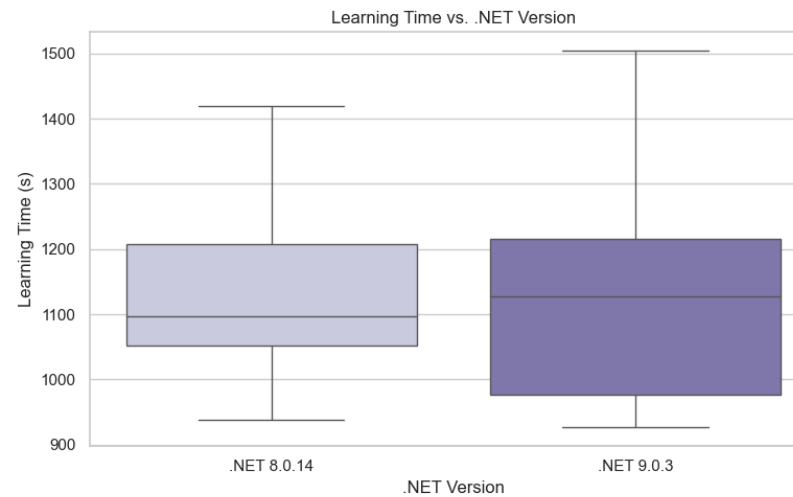
```
private static (double ramUsage, double cpuSpeedGHz, string dotNetVersion) GetSystemPerformanceMetrics()
{
    //double cpuUsage = GetCpuUsage();
    double ramUsage = GetRamUsage();
    double cpuSpeedGHz = GetCpuSpeedGHz();
    string dotNetVersion = GetDotNetVersion();

    return (ramUsage, cpuSpeedGHz, dotNetVersion);
}
```

```
public static void LogPerformance(
    string experimentName,
    InputParameter input,
    string sequenceName,
    int dataSize,
    double learningTimeSeconds,
    //double cpuUsage,
    double ramUsage,
    double cpuSpeedGHz,
    int cores,
    double trainingAccuracy,
    string dotNetVersion)
```

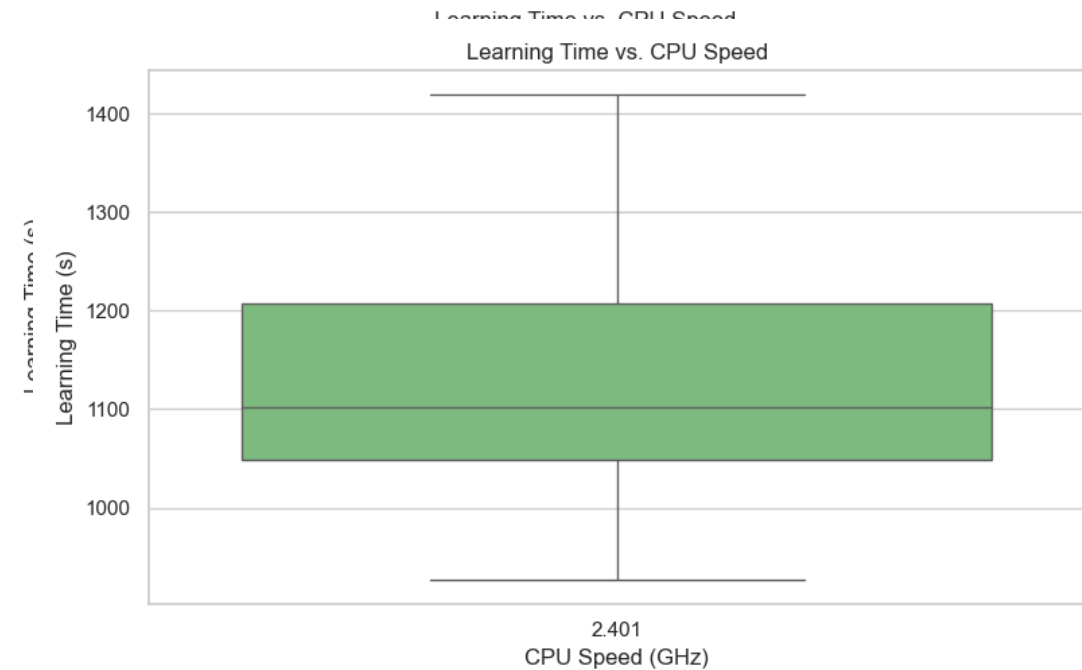
RESULTS

- Learning time vs .NET Version



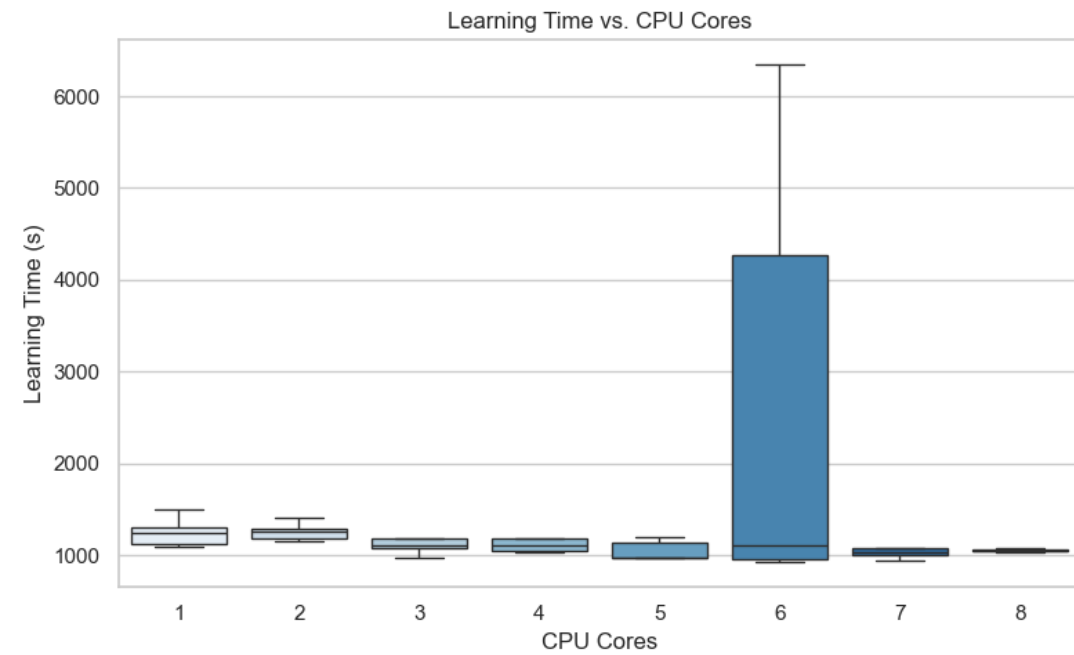
RESULTS

- Learning time vs CPU Speed



RESULTS

- Learning time vs CPU Cores



CONCLUSION

- This project effectively implements a modular and extensible experimental framework for evaluating the effectiveness of learning based on Hierarchical Temporal Memory (HTM), particularly the MultiSequence Learning model. During execution, a console-based runner with configurable parameters allowed for dynamic control over CPU affinity, core selection, and .NET runtime versions.
- Important performance indicators were recorded in CSV outputs, including runtime versions, CPU speed, RAM utilization, and learning time. The experiment's findings supported the benefit of parallelization by demonstrating a correlation between CPU core count and learning efficiency. Additionally, a comparison of the various dotnet versions revealed notable runtime improvements, with .NET 9.0.3 performing better than .NET 8.0.14 in the majority of settings. Because of its reflection-based experiment interface, this program is adaptable enough to be used for more extensive HTM testing and study.

REFERENCES:

- Jeff Hawkins, "On Intelligence: How a New Understanding of the Brain will Lead to the Creation of Truly Intelligent Machines", 2004
- B. H. J. L. R. .Rabiner, "An introduction to hidden Markov models," 1986. [Online]. Available:
http://ai.stanford.edu/~pabbeel/depth_qual/Rabiner_Juang_hmms.pdf
- Yudhi Adhitya, "IoT and Deep Learning-Based Farmer Safety System" 2023.[Online]. Available: https://www.researchgate.net/figure/Hierarchical-Temporal-Memory-HTM-algorithm-flowchart_fig2_369126441
- Keele, "Sequence learning," - B. A. C. G. J. D. S. W., 1998. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/21227209/>

Thank You